VMPrint

A deterministic, zero-DOM typesetting engine/VM in pure TypeScript.

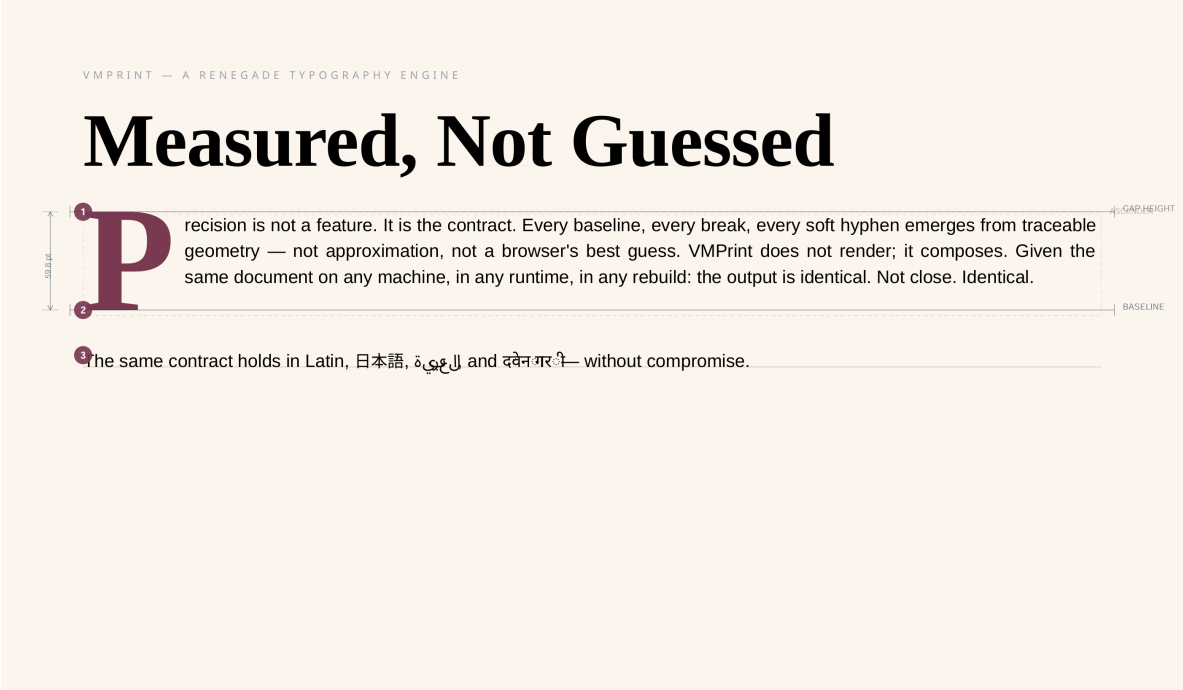
VMPrint is not another HTML-to-PDF wrapper. Instead, it is the crucial missing primitive you might need if you want to build one on your own — from scratch.

If you want to build a custom edge-rendering pipeline, a better document generator, or an entirely new word processor without resorting to contenteditable hacks, hidden iframes, or sloppy DOM overlays — VMPrint provides the low-level mathematical infrastructure to make it happen.

It is a compact, zero-dependency layout Virtual Machine that completely bypasses the browser's HTML/CSS box model. Using interval-arithmetic and a custom morphable-box architecture, it calculates complex print layouts natively. It handles multi-column text wrapping, cross-gutter floats, strict baseline grids, and multilingual line-breaking directly from a semantic JSON AST, outputting a flat array of absolute X/Y coordinates.

Because it operates purely on math and carries zero DOM dependencies, it runs identically everywhere: Cloudflare Workers, Lambda, Bun, Deno, or directly in the browser. It provides the missing primitives for true programmatic document layout.

To see what this engine can build, view the beautifully typeset PDF version of this README[1]. It was compiled directly from this Markdown source file using `draft2final`, a fully-featured manuscript and screenplay compiler built entirely on top of the VMPrint API.



The above image — including annotations, measurement guides, legends, and script direction markers — are all rendered entirely within VMPrint. The source documents can be found in the repository under /documents/readme-assets/.

Features at a Glance

• **Deterministic Layout Engine**: Generates bit-for-bit identical layout across operating systems and runtimes. No more layout drift.

• **Zero Environment Dependencies**: The core engine requires no headless browser, DOM, or Node.js built-ins. Runs seamlessly in browsers, Node.js, and edge environments like Cloudflare Workers and AWS Lambda.

• **True Glyph-Based Measurement**: Reads intrinsic OpenType advance widths and kerning pairs from font files. Layout relies on absolute typographic math, not browser approximations.

• **Fast Performance**: Renders complex, multi-page layouts in milliseconds. Global caches for glyph metrics and text segmentation ensure high throughput for batch pipelines.

• **Multi-Column & Mixed Layouts**: Native support for DTP-style multi-column story regions. Seamlessly mix single-column headers, three-column articles, and pull-quotes on the same page. Floating obstacles naturally shape text across multiple column boundaries.

• **Advanced Pagination & Features**: Floating elements, drop caps, widow/orphan control, and cross-page continuation markers.

• **Complex Table Support**: First-class handling for tables that span multiple pages, including smart row splitting, `colspan`, `rowspan`, and automatically repeated headers.

• **Publishing-Grade Typography**: Grapheme-accurate line breaking using `Intl.Segmenter`, language-aware hyphenation, and mixed-script text runs with perfect baseline alignment.

• **JSON-Based Layout Pipeline**: Layout output is a serializable object tree. Pre-compile layouts into JSON to rapidly distribute identical layouts that render instantly at runtime, snapshot them for CI regression testing, or inspect exact sub-point glyph measurements.

• **Pluggable Architecture**: Swappable font managers and rendering backends (PDF provided). Easily extensible to SVG, Canvas, or custom contexts.

• **Markdown-to-PDF (`draft2final`)**: Includes a declarative Markdown compiler to instantly output formatted PDFs (Screenplay, Academic, Novel, etc.) without writing a single line of layout code.

Baselines, shaping, and directionality remain stable across mixed-language content. *The above image — including annotations, measurement guides, legends,*

and script direction markers — are all rendered within VMPrint. The source documents can be found in the repository under /documents/readme-assets/.

Background

In the 1980s and 90s, serious software thought seriously about pages. TeX got hyphenation and justification right. Display PostScript gave NeXT workstations a real imaging model — every application had access to typographically precise, device-independent layout at the OS level. Desktop publishing software understood widows, orphans, and the subtle difference between a line break and a paragraph break.

Then the web happened. Mostly great. But somewhere along the way, “generate a PDF” became either “run a headless Chromium instance” or “write your own pagination loop against a low-level drawing API.” Neither of these is good. The thinking that went into document composition — the kind that made TeX and PostScript genuinely good — largely disappeared from the toolkit of the working developer.

VMPrint is an attempt to recover some of what was lost.

The Core Idea

VMPrint works in two stages, and keeping them separate is the whole point.

Stage 1 — Layout. You give it a document: structured JSON, or Markdown via `draft2final`. It measures glyphs, wraps lines, handles hyphenation, paginates tables, controls orphans and widows, places floats. It produces a `Page[]` stream — an array of pages, each containing a flat list of absolutely-positioned boxes.

Stage 2 — Rendering. A renderer walks those boxes and paints them to a context. Today that context is a PDF. Tomorrow it could be canvas, SVG, or a test spy.

The `Page[]` stream is the thing that makes this different. It’s serializable JSON. You can diff it between versions. You can snapshot it for regression tests. You can inspect it to understand why

something ended up where it did. Layout bugs become reproducible. This is not how PDF generation usually works.

Layout is based on real font metrics. VMPrint loads actual font files, reads glyph advance widths and kerning data, and measures text the way a typesetting system does — not the way a browser estimates it from computed styles. There is no CSS box model underneath. Same font files, same input, same config: identical output, down to the sub-point position of every glyph.

```
const engine = new LayoutEngine(config, runtime);
await engine.waitForFonts();

// pages is a plain Page[] — inspect it, snapshot it, diff it
const pages = engine.paginate(document.elements);

const renderer = new Renderer(config, false, runtime);
await renderer.render(pages, context);
```

Identical Output, Everywhere

The core engine has no dependency on Node.js, the browser, or any specific JavaScript runtime. It doesn't call `fs`. It doesn't touch the DOM. It doesn't assume `Buffer` exists. Font loading and rendering are injected through well-defined interfaces — the engine itself is pure, environment-agnostic JavaScript.

In practice: run VMPrint in a browser extension, a Cloudflare Worker, a Lambda, and a Node.js server. The layout output is identical. Same page breaks. Same line wraps. Same glyph positions. The rendering context changes; the layout does not.

This is not a promise about "should work in theory." It's an architectural constraint that was enforced from the beginning.

Why Not Just Use...

Headless Chrome / Puppeteer: Works great until it doesn't. Cold starts are slow. Output drifts across browser versions. Edge runtimes typically can't run it at all. You're maintaining a Chromium dependency to produce text in a box — and Chromium is ~170 MB on disk. VMPrint's full dependency tree, including the font engine that makes real glyph measurement possible, is ~2 MiB packed and ~8.7 MiB unpacked.

PDFKit / pdf-lib / react-pdf: You're writing pagination. "If this paragraph doesn't fit, cut here, carry the rest to the next page" — by hand, for every element type, including tables that span pages and headings that must stay with what follows them.

LaTeX: Genuinely excellent at what it does. Also requires a TeX installation, a 1970s input format, and an afternoon of fighting package conflicts.

VMPrint handles the pagination. You describe your document. It figures out where things break.

How It Started

> *I'm a film director. I hated writing in screenplay software, so I started writing in plain text. Then I wrote a book in Markdown and wanted industry-standard manuscript output — and found no tool I trusted to get there without pain.*
>
> *Low-level PDF libraries made me implement my own pagination. Headless browser pipelines were heavy and unpredictable. So I took a detour and built a layout engine first.*
>
> *The manuscript is still waiting. The engine shipped instead.*

Getting Started

Prerequisites: Node.js 18+, npm 9+

```
git clone https://github.com/cosmiciron/vmprint.git
cd vmprint
npm install
```

Render a JSON document to PDF:

```
npm run dev —prefix cli — —input engine/tests/fixtures/regression/00-all
-capabilities.json —output out.pdf
```

Markdown to PDF (screenplay format):

```
npm run dev —prefix draft2final — build draft2final/tests/fixtures/
screenplay-sample.md -o screenplay.pdf —format screenplay
```

Full API Example

```
import fs from ‘fs’;
import { LayoutEngine, Renderer, toLayoutConfig, createEngineRuntime }
from ‘@vmprint/engine’;
import { PdfContext } from ‘@vmprint/context-pdf’;
import { LocalFontManager } from ‘@vmprint/local-fonts’;

const runtime = createEngineRuntime({ fontManager: new LocalFontManager()
 });
const config = toLayoutConfig(documentInput);
const engine = new LayoutEngine(config, runtime);

await engine.waitForFonts();
const pages = engine.paginate(documentInput.elements);

const context = new PdfContext({
  size: [612, 792],
  margins: { top: 0, right: 0, bottom: 0, left: 0 },
  autoFirstPage: false,
  bufferPages: false
});

// Wire the context output to a Node.js write stream.
// The caller owns I/O; the context owns rendering.
const fileStream = fs.createWriteStream(‘output.pdf’);
context.pipe({
  write(chunk) { fileStream.write(chunk); },
  end()        { fileStream.end(); },
  waitForFinish() {
    return new Promise((resolve, reject) => {
      fileStream.once(‘finish’, resolve);
      fileStream.once(‘error’,  reject);
    });
  }
});
```

```
const renderer = new Renderer(config, false, runtime);
await renderer.render(pages, context);
```

To produce a PDF with no embedded fonts — using only the 14 standard PDF fonts that every viewer guarantees — swap in `StandardFontManager`:

```
import { StandardFontManager } from '@vmprint/standard-fonts';

const runtime = createEngineRuntime({ fontManager: new
StandardFontManager() });
```

The rest of the pipeline is identical. The engine detects the sentinel buffers that `StandardFontManager` returns and bypasses fontkit in favor of built-in AFM metrics. The PDF output carries font name references only — no binary font data. See `font-managers/`[2] for details.

What It Can Do

Pagination & Layout

- Desktop Publishing (DTP) style multi-column story regions with adjustable gutters
- Seamless mixed-layout pages (e.g., full-width headers flowing directly into 3-column articles)
- `keepWithNext`, `pageBreakBefore`, orphan and widow controls
- Tables that span pages: `colspan`, `rowspan`, row splitting, repeated header rows
- Drop caps and continuation markers when content splits across pages
- Story zones with text wrapping around complex floating obstacles (even spanning across columns)
- Inline images and rich objects on text baselines

Typography and Multilingual

Most libraries treat international text as an optional concern — get ASCII layout working first, bolt on Unicode support later. VMPrint's text pipeline was built correctly from the start, because the alternative produces subtly wrong output for most of the world's writing systems.

• Text segmentation uses `Intl.Segmenter` for grapheme-accurate line breaking. A grapheme cluster spanning multiple Unicode code points is always treated as a single unit.

• CJK text breaks correctly between characters, without needing spaces.

• Indic scripts are measured and broken as grapheme units, not codepoints.

• Language-aware hyphenation applies per text segment, so a document mixing English and French body text hyphenates each according to its own rules.

• Mixed-script runs — Latin with embedded CJK, inline code within prose — share the same baseline and are measured correctly across font boundaries.

• Two justification modes: space-based (standard for Latin) and inter-character (standard for CJK and some print conventions).

RTL/bidi support is partial today. Full UAX #9-grade bidirectional behavior is a v1.x item.

CJK — Chinese, Japanese, Korean

汉字 (Hànzì) — Chinese characters used for Mandarin, Cantonese, and classical texts: 天地玄黄，宇宙洪荒。日月盈昃，辰宿列张。寒来暑往，秋收冬藏。闰余成岁，律吕调阳。 ^12.8 v3.2 ^12.8 v3.2

日本語 (Nihongo) — Japanese mixes kanji with hiragana and katakana syllabaries: 春はあけぼの。やうやう白くなりゆく山ぎは、少し明かりて、紫だちたる雲の細くたなびきたる。 ^12.8 v3.2 ^12.8 v3.2

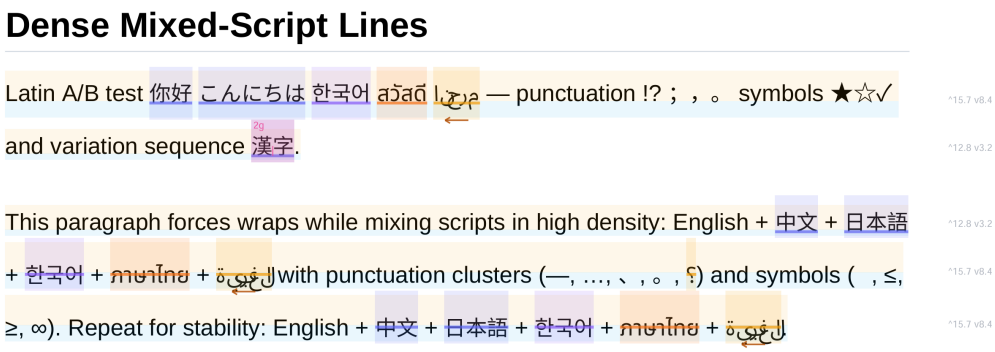
한국어 (Hangugeo) — Korean Hangul syllable blocks, each composed from jamo consonant and vowel units: 이 문서는 다양한 스크립트 시스템이 단일 레이아웃 엔진에서 올바르게 렌더링되는지를 확인합니다. 줄 바꿈과 자간이 안정적이어야 합니다. ^12.8 v3.2 ^12.8 v3.2 ^12.8 v3.2

Variation selectors change glyph form without changing Unicode identity: 漢字 uses a variation-selector sequence. Such sequences must not be split across segments. ^12.8 v3.2 ^10.9 v2.5

Arabic — Right-to-Left Shaping

— اللغة العربية Letters change shape depending on their position within a word (initial, medial, final, or isolated): بِسْمِ اللَّهِ الرَّحْمَٰنِ الرَّحِيمِ ^16.5 v8.9 Note Arabic RTL ^16.5 v8.9

Mixing scripts in a single paragraph — a common requirement in multilingual publishing: The word نشر means "for publishing" and مرحبا means "welcome". Bidirectional text must resolve direction per-run and align glyphs from the correct edge. ^10.9 v2.5 ^15.7 v8.4 ^10.9 v2.5



> ***Multilingual Rendering.*** *The images above — including all annotations, measurement guides, legends, and script direction markers — are rendered entirely by VMPrint. Source document can be found in the repository under* `engine\tests\fixtures\regression`*.*

Architecture

- Core engine is pure TypeScript with zero runtime environment dependencies — no Node.js APIs, no DOM, no native modules
- One codebase runs in-browser, Node.js, serverless, and edge runtimes with identical layout output
- Swappable font managers and rendering contexts via clean interfaces
- Overlay hooks for watermarks, debug grids, and print marks
- Input immutability and snapshot-friendly output for regression testing

Performance & Footprint

VMPrint is built for sustained throughput. The measurement cache, font cache, and glyph metrics cache are all shared across `LayoutEngine` instances that use the same `EngineRuntime` — so batch pipelines get faster as the runtime warms up, not slower.

On a 9-watt low-power i7, the engine's most complex regression fixture — 8 pages of mixed-script typography, floated images, and multi-page tables — completes in:

Scenario	font load	layout	total
Warm (shared runtime, batch pipeline)	~10 ms	~66 ms	~87 ms
Cold (fresh process, first invocation)	~53 ms	~239 ms	~292 ms

The warm figure is what batch PDF generation looks like after the first document has been processed: fonts are already parsed, text measurements are cached, and `paginate()` spends its time on composition rather than measurement. The cold figure is what a fresh CLI invocation sees — fonts parsed from disk, measurement cache empty, JIT compilation running through the hot paths for the first time.

Run the full benchmark suite yourself:

```
cd engine && npm run test:perf — —repeat=5
```

Or profile a specific document with the CLI's `--profile-layout` flag, which runs the document cold then twice more warm and reports both:

```
[vmprint] cold  fontMs: 53.07 | layoutMs: 239.21 | total: 292.28 (8
pages)
[vmprint] warm  fontMs: 0.21  | layoutMs: 68.44  | total: 68.65  (avg ×2)
```

Footprint: The core engine package is measured by npm tarball size (`npm pack --dry-run --json` in `engine/`). At the time of writing this README, `@vmprint/engine` packs to **136,449 bytes (~133 KiB)**. This is distinct from browser bundle size, which depends on bundler target/format and whether code is minified/compressed.

Static standard-font bundle snapshot (2026-03-06):

Artifact	Raw	Gzip	Brotli
`docs/examples/ast-to-pdf` runtime (`index.html` + `styles.css` + `assets/*.js`)	727,383 B (~710 KiB)	227,878 B (~223 KiB)	186,547 B (~182 KiB)
Same runtime + built-in fixtures (`fixtures/*.js`)	3,441,750 B (~3.28 MiB)	2,242,504 B (~2.14 MiB)	2,182,080 B (~2.08 MiB)

The large jump is from `fixtures/14-flow-images-multipage.js`, which embeds a large base64 image payload and is not required for the core runtime path.

NPM packed sizes (2026-03-06, `npm pack --dry-run --json`):

Package	Tarball size	Unpacked size
`@vmprint/engine`	136,449 B	713,077 B
`@vmprint/context-pdf-lite`	5,101 B	20,001 B
`@vmprint/standard-fonts`	3,553 B	11,232 B

The full dependency tree, including `fontkit` for OpenType parsing, is **~2 MiB** packed and **~8.7 MiB** unpacked — versus Chromium's **~170 MB**. The largest single dependency is `fontkit` (~1.1 MiB packed), which is the cost of reading real glyph metrics rather than approximating them from computed styles. Among headless PDF tools, that's not bloat — it's the price of correctness.

Because the pipeline is synchronous and the footprint is minimal, VMPrint can run directly in edge environments (Cloudflare Workers, Vercel Edge, AWS Lambda) where other solutions often exceed memory or cold-start limits. It is fast enough to serve PDFs synchronously in response to user requests, without background job queues.

Packages

This is a monorepo:

Package	Purpose
`@vmprint/contracts`	Shared interfaces
`@vmprint/engine`	Deterministic typesetting core
`@vmprint/context-pdf`	PDF output context
`@vmprint/local-fonts`	Filesystem font loading
`@vmprint/standard-fonts`	Sentinel-based standard font manager (no font assets)
`@vmprint/cli`	`vmprint` JSON → bit-perfect PDF CLI
`@draft2final/cli`	Markdown → bit-perfect PDF compiler

Standalone Browser Bundle (Standard Fonts)

VMPrint also supports a fully static, self-contained browser pipeline:

- `StandardFontManager + Engine + PdfLiteContext`
- No Node.js runtime required at usage time
- No server required (`file://` works)
- Programmatic/batch-friendly in browser or embedded webview contexts

This is demonstrated in `docs/examples/ast-to-pdf/`[3], where AST JSON is rendered directly to downloadable PDF with plain static assets.

Deployable runtime footprint (2026-03-06):

Bundle	Raw	Gzip	Brotli
`index.html` + `styles.css` + `assets/*.js`	727,383 B (~710 KiB)	227,878 B (~223 KiB)	186,547 B (~182 KiB)

This mode uses PDF standard fonts (PDF-14), but the capability is still full VMPrint layout + pagination, not a toy export path. You get the same deterministic engine primitives (flow composition, pagination rules, multi-column behavior, table pagination, inline objects) in a tiny static bundle that runs with no backend.

For many product surfaces, this opens a practical alternative to heavyweight client PDF stacks or hand-authored jsPDF logic: fully client-side PDF generation with predictable output and a small transfer/runtime footprint. It is especially useful for offline-first apps, embedded webviews, hybrid mobile apps, kiosk software, and constrained environments where shipping a browser server/runtime is not realistic.

Tradeoff: you are constrained to PDF-14 coverage. If you need custom fonts, wide Unicode script support, or advanced shaping beyond standard-font coverage, switch to a font-binary workflow (`LocalFontManager`/custom font manager + fontkit path).

Contributing

The monorepo is layered so that getting involved at any depth is straightforward.

Engine (`engine/`): Layout algorithms, pagination, text shaping, the packager system. This is where the hard problems live — and where a well-placed contribution has the most leverage. Regression snapshot tests make it possible to verify that changes haven't broken existing behavior.

Contexts and Font Managers (`contexts/`, `font-managers/`): Concrete implementations of well-defined interfaces. A new context for canvas or SVG output. A font manager that loads from a CDN

or a bundled asset. The contracts are clear, the surface area is contained, and a working implementation is immediately useful to anyone on that platform.

Draft2Final format flavors: New document formats are purely declarative — a flavor module specifies fonts, margins, heading styles, and how Markdown constructs map to VMPrint elements. No pagination code. No layout code. If you know what a correctly formatted academic paper, legal brief, or technical report should look like, you can write a flavor.

Draft2Final format scripts: A single TypeScript file that maps a normalized Markdown AST to `DocumentInput`. The system hands you a structured, source-annotated document; you decide what it becomes. Screenplay, novel, contract, invoice — each is its own file.

```
npm run test —prefix engine
npm run test:update-layout-snapshots —prefix engine
npm run test —prefix draft2final
npm run test:packaged-integration
```

Status

Version `0.1.0`. The core layout pipeline is working and covered by regression fixtures. PDF output is the production-ready path. RTL/bidi support is partial — full Unicode bidirectional behavior is on the roadmap for v1.x.

This is pre-1.0 software. The API may change.

Architecture[4] · Quickstart[5] · Contributing[6] · Testing[7] · Examples[8] · Roadmap[9]

License

Apache 2.0. See LICENSE[10].

References

1. documents/readme-assets/readme.pdf

2. font-managers/README.md

3. docs/examples/ast-to-pdf/README.md

4. documents/ARCHITECTURE.md

5. QUICKSTART.md

6. CONTRIBUTING.md

7. documents/TESTING.md

8. docs/examples/index.html

9. documents/ROADMAP.md

10. LICENSE